

Gauntlet — Joypad Harness Planning Document

A runtime safety harness governing an AI agent that plays a game like a human — it sees the rendered screen and presses controller buttons. The harness decides which inputs are legal, measures progress, carries state in and out, and raises structured alarms; on failure it feeds feedback back, on repeated failure it stops and asks a human. The harness, not the agent, is evaluated.

The four pillars (each separate from the worker)

Pillar	Responsibility
 Guardrails	Declared rules on controller input — allowed buttons, max/min hold, no impossible combos. Checked <i>before</i> execution.
 Checkpoints	Pass/fail from ground-truth state — stack-height-safe, not-game-over, no-new-holes. Persisted to SQLite.
 Material handling	Capture/normalize state, persist every step, replay a run from any frame.
 Alarms	<code>{type, severity, context, recommended_action}</code> — drive escalation and recovery.

The core: behavior changes from feedback

The graded “must.” Loop: `observe → grade checkpoints → on FAIL: feedback hint + alarm → agent adapts → repeat`. A reckless agent stacks up, trips `STACK_HEIGHT_SAFE`, fires `CHECKPOINT_FAILED`; the feedback drives a better placement and it recovers. Three consecutive primary failures escalate to a human `STOP`. The agent decides one action per beat (not per frame), which makes an LLM-in-the-loop real-time feasible. The agent (Claude Haiku 4.5) sees **only the rendered frame** as an image and returns a controller action via structured output — it plays like a human; the referee reads ground-truth state to grade.

Tech stack

Python + FastAPI harness; the browser runs a custom **Tetris** game on a canvas (the video the agent sees) and applies controller inputs. A **WebSocket bridge** carries frame + state in and actions + events out. **SQLite** persists checkpoints for replay. Workers: **scripted** (reckless), **heuristic** (real Tetris AI), **claudio** (vision) — swappable with no harness changes.

Live dashboard (also the deployed URL)

Split screen — game on the left, the four pillars firing on the right (agent intent, checkpoint pass/fail, alarms, guardrail blocks, current worker). Start/Pause, Reset, live worker-swap, and run replay.

Worker independence & escalation

Any worker implementing `decide(state, feedback) -> action` drops in unchanged. Repeated failure trips a `STOP` that asks a human. **Bonus:** swap a recovery worker in live (scripted → heuristic → claude).

Demo: reckless agent stacks → `STACK_HEIGHT_SAFE` fails → `ESCALATE` alarm → human `STOP` → swap to the heuristic agent (plays low/flat, all green) → swap to Claude (vision) → replay from a checkpoint.